Universidad Michoacana de San Nicolás de Hidalgo

Facultad de Ingeniería Eléctrica (FIE)

Laboratorio de Control Analógico I

Práctica No. 4.- Modelos Matemáticos de Sistemas Lineales.

Nombre; David Cortes Cortes

Matricula: 2201300X

Profesor: Dr. Fernando Ornelas Tellez

Fecha de Entrega: 6 de marzo de 2026.

Objetivo.

El objetivo de esta práctica es dar una introducción al uso de algunas funciones de Matlab orientadas al manejo de modelos matemáticos de sistemas lineales en función de transferencia, tales como tf, zpk, zpk2tf, tf2zpk y residue.

Introducción.

Un sistema lineal invariante en el tiempo (SLIT) es aquel que rige por el principio de la superposición. En el dominio del tiempo, estos sistemas se representan mediante ecuaciones diferenciales lineales con coeficientes constantes. Sin embargo, para facilitar su análisis y diseño, se utiliza la Transformada de Laplace, la cual permite transformar dichas ecuaciones en expresiones algebraicas en el dominio de la frecuencia compleja (s).

La relación resultante entre la salida Y(s) y la entrada U(s), asumiendo condiciones iniciales cero, se denomina Función de Transferencia G(s).

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + b_{m-1} s^{m-1} + \cdots + b_1 s + b_0}{a_n s^n + a_{n-1} s^{n-1} + \cdots + a_1 s + a_0}$$

Esta representación no es única, ya que una función de transferencia puede expresarse de diversas maneras según las necesidades del análisis:

- *Potencias descendentes: Como una división de polinomios en s.*
- *Forma factorizada (zpk): Útil para identificar rápidamente los polos, ceros y la ganancia del sistema.*
- *Suma de fracciones parciales: Facilita la implementación del sistema mediane la 1*
- *conexión en paralelo de subsistema más simples.*

Herramientas de software por ejemplo Matlab, a través de su control systema tolbox, proporciona comandos especializados como tf, zpk y residue para manipular y convertir modelos matemáticos entre estas diferentes formas de manera eficiente.

Desarrollo: Funcion fun_transf

/MATLAB Drive/fun_transf.m

```
function G = fun_transf(num, den)
    % Función que despliega una F.T. en sus tres formas básicas
    % Entradas: num = coeficientes del numerador
    %           den = coeficientes del denominador
    % Salida:   G   = función de transferencia en forma tf

    % Forma 1: Potencias descendentes usando el comando tf
    fprintf('--- Forma de Potencias Descendentes ---\n');
    G = tf(num, den);
    G

    % Forma 2: Factorizada (polos, ceros y ganancia) usando tf2zp y zpk
    fprintf('--- Forma Factorizada (zpk) ---\n');
    [z, p, k] = tf2zp(num, den); % Extrae ceros, polos y ganancia
    G_zpk = zpk(z, p, k);        % Construye forma factorizada
    G_zpk

    % Forma 3: Fracciones parciales usando residue
    fprintf('--- Forma de Fracciones Parciales ---\n');
    [r, p_residue, k_residue] = residue(num, den); % r=residuos, p=polos
    for i = 1:length(r)
        fprintf(' %.4f / (s - (%.4f))\n', r(i), p_residue(i));
    end
    % Si existe término directo (grado num >= grado den) lo muestra
    if ~isempty(k_residue)
        fprintf('  Término directo (K): %.4f\n', k_residue);
    end
end
```

Explicación del Código:

En esta para esta se explicará el código de como se hace en el código para obtener las tres formas de una ecuación de transferencia:

- *Primero que hace la función "fun_transf(num, den)" lo que hace es recibir los coeficientes del numerador y denominador de una función de transferencia para luego proseguir con el despliegue de las 3 formas en que no los pide el ejercicio.*
- ***function G = fun_transf(num, den):*** *Primero lo que hace es definir la función en dos entradas para luego definirlo en una sola salida que es G.*
- ***G = tf(num, den);***
  G
  Esta parte del código que usa el comando tf es ara construir la función e transferencia a partir de los vectores de coeficientes que se den en la consola como num y dem.
- ***[z, p, k] = tf2zp(num, den);***
  G_zpk = zpk(z, p, k);
  G_zpk

En esta parte el comando tf2zp es la que extrae los ceros, polos y la ganancia de la función de transferencia para luego se construya la forma factorizada haciendo más evidente los ceros, polos y la ganancia del sistema.

- ***[r, p_residue, k_residue] = residue(num, den);***
  for i = 1:length(r)
  fprintf(' %.4f / (s - (%.4f))\n', r(i), p_residue(i));
  end
  El comando residue lo que hace es que descompone la función en fracciones parciales. Y el vector "r" contiene los numeradores de cada una de la fracción y "p.residue" es la que contiene los polos correspondientes. El ciclo for es lo que hace que se imprima cada fracción con formato "R/(s-P)".

Ejemplo de Ejecución:

Para la ejecución del código se usó la siguiente ecuación de transferencia:

$$G(s) = \frac{s+3}{2s^2+6s+4}$$

La cual al ejecutarla nos dio como resultado de las tres formas:

```
>> num=[1 3];
>> dem=[2 6 4];
>> G = fun_transf(num, dem);
--- Forma de Potencias Descendentes ---

G =

       s + 3
  ---------------
  2 s^2 + 6 s + 4
 |
Continuous-time transfer function.
Model Properties
--- Forma Factorizada (zpk) ---

G_zpk =

   0.5 (s+3)
  -----------
  (s+2) (s+1)

Continuous-time zero/pole/gain model.
Model Properties
--- Forma de Fracciones Parciales ---
 -0.5000 / (s - (-2.0000))
 1.0000 / (s - (-1.0000))
>>
```

Para el segundo ejemplo se usó la siguiente ecuación de transferencia:

$$G(s) = \frac{s+2}{s^2+5s+6}$$

Y como resultado en la consola nos dio:

```
>> num= [1 2];
>> den=[1 5 6];
>> G = fun_transf(num, den);
--- Forma de Potencias Descendentes ---

G =

      s + 2
  -------------
  s^2 + 5 s + 6

Continuous-time transfer function.
Model Properties
--- Forma Factorizada (zpk) ---

G_zpk =

     (s+2)
  -----------
  (s+3) (s+2)

Continuous-time zero/pole/gain model.
Model Properties
--- Forma de Fracciones Parciales ---
 1.0000 / (s - (-3.0000))
 0.0000 / (s - (-2.0000))
>>
```

Análisis del Circuito RLC

```
/MATLAB Drive/FT_VcVi.m
function G = FT_VcVi(R, L, C)
    % Función que obtiene la F.T. Vc(s)/Vi(s) de un circuito RLC
    % donde R y C están en paralelo, y L en serie con esa combinación
    % Entradas: R = resistencia (Ohms)
    %           L = inductancia (Henrios)
    %           C = capacitancia (Faradios)
    % Salida:   G = función de transferencia Vc(s)/Vi(s)

    % Numerador: R (resultado del divisor de voltaje)
    num = [R];

    % Denominador: LRCs^2 + Ls + R (obtenido por divisor de voltaje)
    den = [L*R*C, L, R];

    % Crear función de transferencia con el comando tf
    G = tf(num, den);
    fprintf('F.T. Vc/Vi para R=%.2f, L=%.2f, C=%.2f:\n', R, L, C);
    G
end
```

Explicación del código:

En esta parte se logra la explicación del código para que se sepa cómo es que funciona cada uno de su comando y se pueda entender mejor su ejecución:

- ***function G = FT_VcVi(R, L, C):*** *Define la función que recibe tres parámetros de entrada (Resistencia, Inductancia y Capacitancia) y entrega como salida el objeto de la función de transferencia G.*
- ***num = [R]:*** *Define el numerador de la función de transferencia. En el análisis del circuito por divisor de voltaje, el término independiente en el numerador corresponde al valor de la resistencia R cuando se simplifica la expresión.*
- ***den = [L*R*C, L, R]:*** *Define el vector de coeficientes del denominador en potencias descendentes de* $\left(s^2,\ s^1,\ s^{(0)}\right)$. *Estos valores surgen de la ecuación característica del circuito:* $LRCs^2 + Ls + R$
- ***G = tf(num, den):*** *Utiliza el comando básico tf para construir el modelo matemático del sistema lineal a partir de los vectores de coeficientes calculados anteriormente.*
- ***fprintf(...) y G:*** *Se encarga de imprimir en la consola los valores de los componentes utilizados para la prueba y muestra visualmente la función de transferencia resultante.*

Ejecución del Código:

Para este código al momento de ejecutarlo que uso la ecuación de transferencia:

$$G(s) + \frac{1}{0.05s^2 + 0.5s + 1}$$

Los valores de R, L y C son los siguientes: R=1, L=0.5 y C=0.10, lo cual al ejecutarlo queda de la siguiente manera en la consola:

```
>> G = FT_VcVi(1, 0.5, 0.1)
F.T. Vc/Vi para R=1.00, L=0.50, C=0.10:

G =

           1
  --------------------
  0.05 s^2 + 0.5 s + 1

Continuous-time transfer function.
Model Properties

G =

           1
  --------------------
  0.05 s^2 + 0.5 s + 1

Continuous-time transfer function.
Model Properties
>>
```

Conclusión:

La presente práctica permitió comprender y aplicar de manera efectiva las herramientas fundamentales que ofrece Matlab para el análisis y representación de sistemas lineales mediante funciones de transferencia. A través del desarrollo de la función fun_transf y el uso de comandos como tf, zpk, tf2zp y residue, se logró expresar un mismo sistema en distintas formas matemáticas. Esto destacó cómo cada representación facilita un aspecto diferente del análisis, ya sea la identificación de polos y ceros, la construcción del modelo algebraico o la obtención de fracciones parciales.

Asimismo, el estudio del circuito RLC a través de la función FT_VcVi reforzó la relación entre los parámetros físicos de un sistema eléctrico y su modelado matemático. Se pudo observar cómo los valores de resistencia, inductancia y capacitancia determinan el comportamiento dinámico del sistema al integrarse en la ecuación característica $LRCs^2 + Ls + R$. En conjunto, la práctica fortaleció la comprensión del modelado de sistemas lineales y el uso de herramientas computacionales esenciales para el análisis y diseño de sistemas de control.